CSE 211 — Theory of Computation

Decidability, Recognisability and Undecidability

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Atif Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — a 15-mark undecidability proof in every paper

Lesson 19 — Recursive, RE and Decidable Languages — definitions, the three classes, decidable problems

Lesson 20 — Counting Arguments, TM Codes and the Diagonalization Language — countable/uncountable, encoding, L_d is not RE

Lesson 21 — Complements, the Universal Language, the Halting Problem, A_TM — L_u, universal TM, halting, A_TM

Lesson 22 — Reductions, Rice's Theorem and Other Undecidable Problems — reduction proofs, TM-property problems, PCP, CFG problems

Quick Notes — Undecidability — revision bullets + key diagrams

NOTE Slide numbers refer to the PDF pages of Lecture - 7 (Undecidability).pdf (77 pages). Notation: L_d (diagonalization language), L_u (universal language) and the halting language follow the slides (Hopcroft); A_TM = {<M, w> | M accepts w} is Sipser's name for L_u in the older papers. The binary code of the slides' example TM, the shortlex enumeration and the decidability constructions below were checked by program. Practice provenance: Actual = taken from a TF paper; Adapted = TF question with changed details; Constructed = new question in TF style.

Exam Map — What the TF Papers Ask

Session	Decidability / undecidability questions
2023-24	Q11 Example of a non-RE language; prove halting is not recursive (Lu not recursive given) (15)
2022-23	Q12 Diagonalization language; prove it is not RE (15)
2021-22	Q11 Diagonalization and universal languages; show halting is not recursive (15)
2020 (Jan term)	Q11 Define RE and recursive languages; example of non-RE and of RE-but-not-recursive (15)
2019-20	Q11 One example each: regular; CF not regular; recursive not CF; RE not recursive; not RE (15) · Q12 Halting problem; prove it is RE but not recursive (15)
2017-18	Q11 Define diagonalization language, universal language, halting problem; show one is not RE (15)
2016-17	Q7(a) Is {<G> | G connected undirected graph} decidable? Design + example (15) · Q8(a) Prove A_TM is undecidable (15)
2015-16	Q7(b) P undecidable: use P and reduction to prove Q undecidable (10⅓)
2014-15	Q7(b) The set of infinite binary sequences is not countable (7) · Q8(a) Decidable, Turing-recognizable and undecidable languages (7) · Q8(c) Prove A_TM is undecidable (9⅓)
2013-14	Q3(b) A = {s}, s = 0/1 depending on life on Mars: is A decidable? (7) · Q4(a) Prove A_DFA = {<B,w> | B is a DFA accepting w} is decidable (7) · Q4(b) Prove the set of rational numbers is countable (7) · Q4(c) Prove A_TM is undecidable (9⅓)
2012-13	Q2(a) Recognizable vs decidable; show HALT is Turing-acceptable (10)
2011-12	Q1(c) An encoding technique for TMs; show the number of TMs is countable (8) · Q2(c) Show HALT = {<M,w> | M halts on w} is Turing-acceptable (7) · Q3(b) Prove: the union of two RE languages is RE (8) · Q4(c) Show {<A,B,C> | A,B,C DFAs and L(C) = L(A) ∪ L(B)} is decidable (8⅓)
2010-11	Q10(a) Show the set of all infinite binary sequences is uncountable (4) · Q10(b) Using 10(a), prove that some languages are not Turing-recognizable (8⅓) · Q10(c) L is decidable iff L and its complement are Turing-recognizable (7) · Q10(d) Formal description of the halting problem; a language that is not Turing-recognizable (4) · Q12(a) Turing-decidable vs Turing-recognizable; the three conditions for M to recognize w (4)
2009-10	Q13(a) 'The operation of a TM may never stop' — agree? Example (4) · Q13(b) Prove: L recursive ⇒ complement of L recursive (4) · Q13(c) Universal Turing machine with an example (11) · Q13(d) The halting problem with an example (4⅓)
2008-09	Q4(a) Describe and sketch a proof of Rice's theorem (12) · Q4(b) Is 'the language accepted by a TM is palindromic' decidable? (3⅓) · Q4(c) Example of a language that is RE but not recursive (3) · Q4(d) Prove: if L and its complement are both RE then both are recursive (5)
2007-08	Q12 Define the diagonalization language; prove it is not RE (10)
2006-07	Q3(c) Enumeration of binary strings; find the 13th binary string (5⅓) · Q4(a) Prove the diagonalization language is not recursively enumerable (10) · Q4(c) Encode the TM M = ({q1,q2,q3},{0,1},{0,1,B},δ,q1,B,{q2}) with the given 4 rules (5⅓)

EXAM FOCUS Every paper has one 15-mark proof question from this lecture: 'diagonalization language is not RE' (2022-23, 2017-18, 2007-08, 2006-07), 'halting problem is RE but not recursive / not recursive' (2023-24, 2021-22, 2019-20), 'A_TM is undecidable' (2016-17, 2014-15, 2013-14), definitions + one example each of regular, CF-not-regular, recursive-not-CF, RE-not-recursive, non-RE languages (Jan-2020, 2019-20). Older papers add countability, TM encoding, Rice's theorem, 'if L and its complement are RE then L is recursive'. The Venn picture of Lesson 21 answers half of them.

Lesson 19 — Recursive, RE and Decidable Languages

Slides 2–13

1. History in One Paragraph (sl. 2–6)

Hilbert's 10th problem (1900): decide whether a polynomial has integral roots — shown algorithmically unsolvable by Matiyasevich (1970). Gödel's incompleteness theorem (1931). Church (λ-calculus) and Turing (machines) in 1936 proved λ-computable = Turing computable = general recursive; the **Church–Turing thesis** says every general way of computing yields only these functions (unprovable since 'general way to compute' is informal). Undecidable problems include the halting problem, ambiguity of CFGs, equality of two CFGs, the mortal-matrix problem (sl. 6).

2. The Classes of Languages (sl. 7–10)

Class	Definition	TM behaviour
Recursively enumerable (RE) = Turing-recognisable	L = L(M) for some TM M	accepts every w ∈ L; for w ∉ L it may reject OR run forever
Recursive = decidable	L = L(M) for a TM M that halts on every input	accepts w ∈ L and halts; halts without accepting for w ∉ L
Undecidable	not recursive	no TM halts on all inputs and accepts exactly L
Not RE	no TM accepts L at all	—

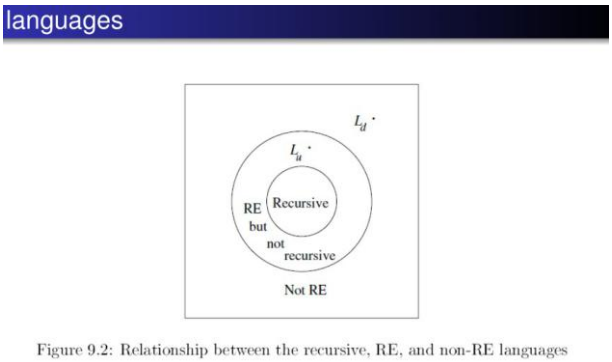


Figure 9.2: Relationship between the recursive, RE, and non-RE languages

Figure 19.1 — Recursive ⊂ RE ⊂ all languages; non-RE outside (sl. 10)

EXAM-READY ANSWER Decidable, Turing-recognisable and undecidable languages (2014-15 Q8(a), 2010-11 Q12(a)): a language L is **Turing-recognisable** if some TM M accepts exactly the strings of L — on w ∈ L it halts and accepts; on w ∉ L it halts in a non-accepting state or loops forever. It is **decidable** (recursive) if some TM decides it: it halts on every input, accepting exactly the members. A language is **undecidable** if no such always-halting TM exists. Every decidable language is recognisable; the converse fails (L_u, the halting problem); and some languages are not even recognisable (L_d). Three conditions for 'M recognises w': (1) if M accepts w, w ∈ L; (2) if w ∈ L, M accepts w; and nothing is required when w ∉ L except that M does not accept (it may loop).

3. Decidable Languages (sl. 11–13)

Problem about	Decidable (a TM halts on all inputs)	Undecidable
Regular languages	A_DFA = {<B,w> | B a DFA accepting w}; A_NFA; A_REX; E_DFA = {<A> | L(A) = ∅}; EQ_DFA	—
Context-free languages	A_CFG = {<G,w> | G generates w} (CYK); E_CFG = {<G> | L(G) = ∅}	EQ_CFG, ambiguity, G ⊆ H, G = regular R, …

Problem about	Decidable (a TM halts on all inputs)	Undecidable
Turing machines	—	A_TM, HALT, E_TM, EQ_TM, 'L(M) is regular / CF / finite' (all language properties)

4. Proofs by Construction of a Decider

EXAM-READY ANSWER A_DFA is decidable (2013-14 Q4(a)): M = 'On input <B, w>, where B is a DFA and w a string: (1) check that <B,w> is a proper encoding (halt and reject otherwise); (2) simulate B on w, keeping the current state of B and the current position in w on the tape (B's description is finite and each step is a table look-up); (3) if B ends in an accepting state, accept; otherwise reject.' The simulation takes exactly |w| steps, so M halts on every input and L(M) = A_DFA. (A_NFA and A_REX: convert to a DFA first — Hakim Lec 4 and 5 — and run the decider.)

EXAM-READY ANSWER Decidability of {<A, B, C> | A, B, C DFAs and L(C) = L(A) ∪ L(B)} (2011-12 Q4(c)): build a DFA D for the symmetric difference L(C) △ L(A ∪ B) using the product construction (accept pairs where exactly one of C and the union-product accepts); then L(C) = L(A) ∪ L(B) iff L(D) = ∅. E_DFA is decidable: mark the start state, repeatedly mark states reachable by an arrow from a marked state until nothing changes; accept iff no accepting state is marked. So the TM 'construct D, run the E_DFA decider on <D>, accept iff it accepts' decides the language (every step terminates).

EXAM-READY ANSWER Is A = {s} decidable? (2013-14 Q3(b)) YES. A is a finite language (one string), hence regular, hence decidable. Exactly one of the two TMs 'accept exactly 0' and 'accept exactly 1' decides A. The question whether life will be found on Mars has an unambiguous answer, so s is a fixed string — we may not KNOW which TM is the right one, but a decider exists; decidability is about existence of the algorithm, not our knowledge of which one it is.

EXAM-READY ANSWER Is A = {<G> | G is a connected undirected graph} decidable? (2016-17 Q7(a)) YES. Encode G as a list of nodes and edges. Decider: select the first node and mark it; repeat: for every edge, if exactly one endpoint is marked, mark the other, until no new node is marked; if all nodes are marked accept, otherwise reject. At most n rounds, each polynomial — it always halts. Example: nodes {1,2,3,4}, edges (1,2), (2,3): mark 1 → 2 → 3; node 4 stays unmarked ⇒ reject; adding (3,4) ⇒ all marked ⇒ accept. (Checked by program.)

EXAM-READY ANSWER 'The operation of a TM may never stop' (2009-10 Q13(a)): True. A TM on w ∉ L may loop forever, e.g. δ(q0, 0) = (q0, 0, R), δ(q0, B) = (q0, B, R) on any input never halts. This is exactly why recognisable (RE) is a weaker notion than decidable, and why 'will M halt on w?' (the halting problem) is hard.

Practice — Lesson 19

(a) Actual — 2014-15 Q8(a), 2010-11 Q12(a). (b) Actual — 2013-14 Q4(a). (c) Actual — 2011-12 Q4(c). (d) Actual — 2013-14 Q3(b). (e) Actual — 2016-17 Q7(a). (f) Actual — 2009-10 Q13(a). (g) Actual — 2007-08 Q11, 2020 Q11.

(a) What do you mean by decidable, Turing recognisable and undecidable languages? (7)

(b) Prove that A_DFA = {<B, w> | B is a DFA accepting w} is decidable. (7)

(c) Show that {<A,B,C> | A, B, C are DFAs and L(C) = L(A) ∪ L(B)} is decidable. (8⅓)

(d) A = {s}, s = 0 if life will never be found on Mars, 1 if it will be found someday. Is A decidable? Why? (7)

(e) A = {<G> | G is a connected undirected graph}. Is it possible to design a TM that decides A? If yes briefly discuss the design and demonstrate with an example. (15)

(f) 'The operation of a Turing machine may never stop.' Do you agree? Give an example. (4)

(g) What are recursive, RE and non-RE languages? (6)

✓ Solution

(a), (b)–(f) EXAM-READY ANSWERS above (the proofs, the Mars answer, the graph decider with its example, the looping TM).

(g) Recursive = decidable (§2 table); RE = recognised by a TM that may loop on non-members; non-RE = recognised by no TM at all — e.g. L_d (Lesson 20) and the complement of L_u (Lesson 21).

Lesson 20 — Counting Arguments, TM Codes and the Diagonalization Language

Slides 14–51

1. Countable and Uncountable (textbook counting arguments)

EXAM-READY ANSWER The rational numbers are countable (2013-14 Q4(b)): a set is countable if there is a bijection with $\mathbb{N}$ (or it is finite). Every positive rational is p/q with p, q ≥ 1; arrange them in an infinite table, row p, column q, and list the table along its diagonals p + q = 2, 3, 4, … (1/1; 2/1, 1/2; 3/1, 2/2, 1/3; …), skipping fractions already listed (like 2/2). Every positive rational appears after finitely many steps, so we get a bijection with $\mathbb{N}$. Negative rationals and 0 are interleaved (0, 1, −1, 1/2, −1/2, …). Hence $\mathbb{Q}$ is countable (the same dovetailing idea lists the IDs of an NTM in Lesson 17).

EXAM-READY ANSWER The set of infinite binary sequences is NOT countable (2010-11 Q10(a), 2014-15 Q7(b)) — Cantor's diagonal argument. Suppose the sequences could be listed s1, s2, s3, … (s_i = b_i1 b_i2 b_i3 …). Define the sequence d whose i-th bit is the complement of the i-th bit of s_i: d_i = 1 − b_ii. Then d differs from every s_i in position i, so d is not in the list — contradiction. Hence no such enumeration exists.

EXAM-READY ANSWER Some languages are not Turing-recognisable (2010-11 Q10(b)). (i) $\Sigma*$ is countable (strings listed in shortlex order), and every TM has a finite description — a string over a finite alphabet — so the set of all TMs is countable, and therefore so is the set of RE languages. (ii) The set of all languages over Σ is the power set of $\Sigma*$; each language corresponds to an infinite binary sequence (its characteristic sequence over the shortlex list of $\Sigma*$), and these sequences are uncountable by Cantor's argument. Hence there are uncountably many languages but only countably many TMs, so some language is not recognised by any TM. The number of TMs is countable because each TM can be encoded as a binary string (§2) and binary strings are countable (2011-12 Q1(c)).

2. Enumerating Strings and Coding Turing Machines (sl. 17–39)

Enumerating binary strings (sl. 17): treat 1w as a binary integer i and call w the i-th string w_i: ε is the 1st, 0 the 2nd, 1 the 3rd, 00 the 4th, 01 the 5th, …, i.e. by length, then lexicographically. The 13th binary string: $13 = 1101_2$, so w_13 = 101 (list: ε, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, 101).

Coding a TM M = ({q1,…,qr}, {0,1}, {X1,…,Xs}, δ, q1, B, {q2}) as a binary string (sl. 18–24): q1 start, q2 the only accepting state (the machine halts when it accepts), X1 = 0, X2 = 1, X3 = B, other symbols arbitrary, D1 = L, D2 = R. A rule δ(q_i, X_j) = (q_k, X_l, D_m) is coded 0^i 1 0^j 1 0^k 1 0^l 1 0^m — no two consecutive 1s inside. The whole machine is c1 11 c2 11 … 11 cn (the rules in any order, separated by 11).

Rule	Code
δ(q1, 1) = (q3, 0, R)	0 1 00 1 000 1 0 1 00 = 0100100010100
δ(q3, 0) = (q1, 1, R)	000 1 0 1 0 1 00 1 00 = 0001010100100
δ(q3, 1) = (q2, 0, R)	000 1 00 1 00 1 0 1 00 = 00010010010100
δ(q3, B) = (q3, 1, L)	000 1 000 1 000 1 00 1 0 = 0001000100010010

```
Code of M (2006-07 Q4(c), 2011-12 Q1(c); slides sl. 25–39):
0100100010100110001010100100110
0010010010100110001000100010010
```

There are 4! = 24 codes of this M (rule order) — and many more with other state/symbol numberings; this does not matter (sl. 39). A string that does not begin with 0 or has three consecutive 1s is not a code; for those i the machine M_i is the TM with one state and no transitions (it halts at once, L(M_i) = ∅) (sl. 40–41).

EXAM-READY ANSWER An encoding technique and countability (2011-12 Q1(c)): number states and symbols by integers, code each move δ(q_i, X_j) = (q_k, X_l, D_m) as 0^i10^j10^k10^l10^m, and the TM as c1 11 c2 11 … cn. This gives each TM at least one binary string, and different TMs get different strings; since the set of binary strings is countable (listed in shortlex order), so is the set of TMs: we can speak of 'the i-th TM M_i' — the TM whose code is the i-th binary string w_i.

3. The Diagonalization Language L_d (sl. 40–51)

L_d = {w_i | w_i ∉ L(M_i)}: all strings w such that the TM whose code is w does not accept w. Picture a table with rows M_i and columns w_j; entry 1 if M_i accepts w_j. The diagonal says whether M_i accepts w_i; L_d's characteristic vector is the **complemented diagonal** (1, 0, 0, 0, … begins: ε ∈ L_d; 0, 1, 00 ∉ L_d), which differs from row i in position i for every i (sl. 43–46). The barber of Seville ('shaves exactly those who do not shave themselves') is the same paradox (sl. 47).

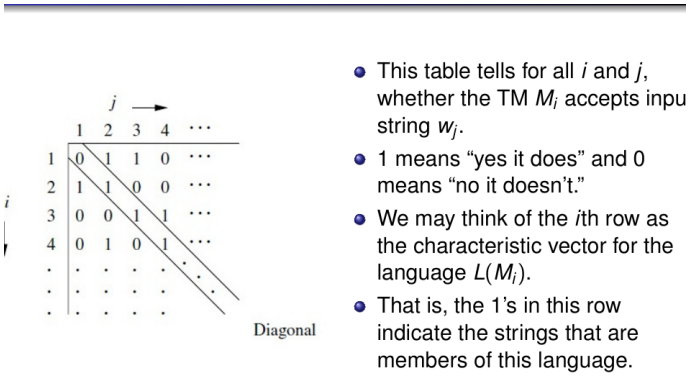


Figure 20.1 — The table 'M_i accepts w_j' and its diagonal (sl. 44)

EXAM-READY ANSWER Theorem: L_d is not recursively enumerable (2022-23 Q12, 2017-18 Q11, 2007-08 Q12, 2006-07 Q4(a); sl. 48–51). Proof by diagonalization. Suppose L_d = L(M) for some TM M. L_d is a language over {0,1}, so M has an input alphabet {0,1} and is in our list: M = M_i for some i (it has a code w_i). Ask whether w_i is in L_d. If w_i ∈ L_d then M_i accepts w_i (since L(M_i) = L_d) — but by the definition of L_d, w_i ∈ L_d means M_i does NOT accept w_i. Contradiction. If w_i ∉ L_d then M_i does not accept w_i — but then by the definition of L_d, w_i ∈ L_d. Contradiction again. So w_i can be neither in nor out of L_d, and the assumption that a TM M accepting L_d exists is false: L_d is not RE (in particular it is not recursive).

Practice — Lesson 20

(a) Actual — 2006-07 Q4(c). (b) Actual — 2006-07 Q3(c). (c) Actual — 2022-23 Q12 / 2007-08 Q12 / 2006-07 Q4(a). (d) Actual — 2010-11 Q10(a)–(b). (e) Actual — 2013-14 Q4(b). (f) Actual — 2011-12 Q1(c).

(a) Encode the TM M = ({q1,q2,q3}, {0,1}, {0,1,B}, δ, q1, B, {q2}) with δ(q1,1) = (q3,0,R), δ(q3,0) = (q1,1,R), δ(q3,1) = (q2,0,R), δ(q3,B) = (q3,1,L). (5⅓)

(b) Explain how binary strings are enumerated; find the 13th binary string. (5⅓)

(c) Define the diagonalization language. Prove that it is not recursively enumerable. (10–15)

(d) Show that the set of all infinite binary sequences is uncountable; use it to prove that some languages are not Turing-recognisable. (4 + 8⅓)

(e) Prove that the set of rational numbers is countable. (7)

(f) Describe an encoding technique for TMs and show that the number of TMs is countable. (8)

✓ Solution

(a) Table and code in §2 (the whole code, 62 bits, computed and compared with the slide's code by a program).

(b) §2: w_13 = 101.

(c)–(f) EXAM-READY ANSWERS in §1 and §3.

Lesson 21 — Complements, the Universal Language, the Halting Problem, A_TM

Slides 52–70

1. Complements (sl. 52–57)

EXAM-READY ANSWER If L is recursive so is its complement (2009-10 Q13(b); sl. 52–53). Let L = L(M) with M halting on every input. Build M′: make the accepting states of M non-accepting with no transitions (M′ halts there without accepting); add a new accepting state r with no moves; for every non-accepting state q and tape symbol X such that M has no move (M halts rejecting), add δ(q, X) = (r, X, R). Since M always halts, so does M′, and M′ accepts exactly the strings M rejects: L(M′) = complement of L.

EXAM-READY ANSWER If L and its complement are both RE then L is recursive (2008-09 Q4(d), 2010-11 Q10(c), 2011-12 Q3(b)-style; sl. 54–56). Let L = L(M1) and complement(L) = L(M2). Build a two-tape TM M simulating M1 on tape 1 and M2 on tape 2 'in parallel' (alternately one step each; states are pairs). If w ∈ L, M1 eventually accepts and M accepts and halts. If w ∉ L, then w ∈ complement(L), M2 eventually accepts, and M halts without accepting. So M halts on every input and L(M) = L: L is recursive (hence so is its complement). Consequence: the complement of an RE-but-not-recursive language is not RE.

EXAM-READY ANSWER The union of two RE languages is RE (2011-12 Q3(b)): run M1 and M2 alternately, one step each (two tapes); accept as soon as either accepts. (Intersection: run both, accept when both have accepted. The RE languages are NOT closed under complement, by the previous result and Lesson 21 §2.)

Since L_d is not RE, its complement cannot be recursive; in fact the complement of L_d is RE but not recursive (sl. 57).

2. The Universal Language L_u and the Universal TM (sl. 58–66)

L_u = {<M, w> | M a TM over {0,1}, w ∈ {0,1}∗, w ∈ L(M)} — the codes of pairs (M, w) with w accepted by M (in Sipser's notation A_TM). **Universal Turing machine U** (multitape, sl. 59–63): tape 1 holds the code of M, then 111, then w; tape 2 is M's simulated tape (symbol X_i written 0^i, separated by single 1s); tape 3 holds M's state (state q_i as i zeros). U (1) checks the code is legitimate (else halts without accepting), (2) writes w in coded form on tape 2, (3) puts the start state 0 on tape 3 and the head of tape 2 on the first simulated cell, (4) to simulate a move searches tape 1 for a rule 0^i10^j10^k10^l10^m with 0^i = the state on tape 3 and 0^j = the symbol under the simulated head, then changes tape 3 to 0^k, replaces 0^j on tape 2 by 0^l (shifting if lengths differ), moves the head to the next 1 on the left (m = 1) or right (m = 2); (5) if no rule matches, M has halted: U halts; (6) if M enters its accepting state, U accepts. Hence L(U) = L_u: L_u is RE.

EXAM-READY ANSWER L_u is RE but not recursive (sl. 64–66). RE: the universal TM U. Not recursive: suppose L_u were recursive; then its complement would be recursive, and a TM M for it (a decider for 'M_i does not accept w_i') can be turned into a TM M′ for L_d: on input w, M′ changes the input to w111w and runs M on it. If w = w_i, M′ accepts iff <M_i, w_i> ∈ complement(L_u), i.e. iff M_i does not accept w_i, i.e. iff w_i ∈ L_d. So L(M′) = L_d, contradicting the fact that L_d is not RE. Hence L_u is not recursive.

3. The Halting Problem (sl. 67–70)

H(M) = the set of inputs on which M halts (accepting or not); HALT = {<M, w> | w ∈ H(M)} (Turing's original machines accepted by halting).

EXAM-READY ANSWER HALT is RE but not recursive (2019-20 Q12, 2021-22 Q11, 2023-24 Q11, 2012-13 Q2(a), 2011-12 Q2(c), 2009-10 Q13(d); sl. 67–70). RE ('Turing-acceptable'): U_H simulates M on w exactly like the universal TM, but accepts whenever the simulated M halts (accepting or rejecting); if M never halts, U_H runs forever. So U_H accepts HALT. NOT recursive: suppose a TM H decides HALT. Then we could decide L_u: on <M, w> run H on <M, w>; if H says M does not halt, reject; if H says M halts, simulate M on w (it halts after finitely many steps) and accept iff M accepts. This decides L_u — impossible (L_u is not recursive). Hence the halting problem is undecidable ('we have reduced L_u to HALT', sl. 70).

4. A_TM is Undecidable — Two Proofs (2013-14 Q4(c), 2014-15 Q8(c), 2016-17 Q8(a))

EXAM-READY ANSWER A_TM = {<M, w> | M is a TM that accepts w} is undecidable. Proof 1 (diagonal, Sipser). Suppose a TM H decides A_TM: H(<M, w>) accepts if M accepts w and rejects otherwise (and always halts). Define D: 'On input <M>, run H on <M, <M>>; output the OPPOSITE of H's answer (accept if H rejects, reject if H accepts).' D is a TM that halts on every input. Now run D on its own description <D>: D accepts <D> ⇔ H rejects <D, <D>> ⇔ D does not accept <D>. Contradiction; so H cannot exist. Proof 2 (via L_u): A_TM is the universal language L_u, which is not recursive by §2 (otherwise L_d, whose complement is the diagonal of the table, would be RE).

EXAM-READY ANSWER Examples of each kind (Jan-2020 Q11, 2019-20 Q11; classes sl. 7–13): regular: (01)∗ (finite memory); context-free but not regular: {0^n1^n | n ≥ 0}; recursive (decidable) but not context-free: {0^n1^n2^n | n ≥ 0} (a TM decides it, Atif TM note); RE but not recursive: L_u (A_TM) or HALT; not RE: L_d or the complement of A_TM. Chain: regular ⊂ CF ⊂ recursive ⊂ RE ⊂ all languages.

Practice — Lesson 21

(a) Actual — 2009-10 Q13(b). (b) Actual — 2008-09 Q4(d), 2010-11 Q10(c). (c) Actual — 2011-12 Q3(b). (d) Actual — 2009-10 Q13(c)–(d). (e) Actual — 2019-20 Q12, 2023-24 Q11, 2021-22 Q11, 2012-13 Q2(a), 2011-12 Q2(c). (f) Actual — 2013-14 Q4(c), 2014-15 Q8(c), 2016-17 Q8(a). (g) Actual — Jan-2020 Q11, 2019-20 Q11, 2017-18 Q11, 2008-09 Q4(c). (h) Actual — 2010-11 Q10(d).

(a) Prove: if L is a recursive language, so is its complement. (4)

(b) Prove: if both L and its complement are RE then L is recursive. (5–7)

(c) Prove that the union of two RE languages is RE. (8)

(d) Describe the universal Turing machine with an example; describe the halting problem. (11 + 4⅓)

(e) Prove that the halting problem is RE but not recursive (you may assume L_u is not recursive). (15)

(f) Prove that A_TM is undecidable. (9⅓ / 15)

(g) Define RE and recursive languages; give one example each of a language that is (i) not RE, (ii) RE but not recursive; or (one example each) regular, CF not regular, recursive not CF, RE not recursive, not RE. (15)

(h) Give the formal description of the halting problem and a language that is not Turing-recognisable. (4)

✓ Solution

(a)–(f) EXAM-READY ANSWERS in §1–§4.

(d) UTM with an example: take M with code c and input w = 01; U writes 111 01 after c on tape 1 and copies the coded w on tape 2; then it repeatedly finds the matching rule 0^i10^j10^k10^l10^m: with the Lesson-20 M, start state 0, scanning 1 = 00: rule 0 1 00 1 000 1 0 1 00 matches ⇒ state becomes 000, write 0, move right (m = 2) … and so on until q2 (00) is entered ⇒ accept.

(g) Definitions: Lesson 19 §2; examples: EXAM-READY ANSWER at the end of §4. Non-RE: L_d (§3 of Lesson 20); RE but not recursive: L_u / HALT.

(h) HALT = {<M, w> | M halts on input w}; a non-RE language: L_d, or the complement of HALT/A_TM.

Lesson 22 — Reductions, Rice's Theorem and Other Undecidable Problems

Slides 71–77

1. Reductions (sl. 71)

If an algorithm converts every instance of P1 into an instance of P2 so that the answer to P1 can be read from the answer to P2, then **P1 reduces to P2**, and P2 is at least as hard as P1: if P1 is not recursive, neither is P2; if P1 is non-RE, neither is P2. (Direction: reduce the KNOWN undecidable problem TO the new one.)

> **EXAM-READY ANSWER** Proving Q undecidable from a known undecidable P (2015-16 Q7(b)): Suppose Q were decidable by a TM R. Give a computable transformation f that maps every instance x of P to an instance f(x) of Q such that $x \in P \Leftrightarrow f(x) \in Q$. Then the TM 'on x compute f(x) (this halts), run R on f(x), and give R's answer' would decide P. But P is undecidable — contradiction; so Q is undecidable. Example: HALT is undecidable because A_TM reduces to it (Lesson 21 §3); E_TM is undecidable because from <M, w> we build M_w that erases its input, simulates M on w and accepts iff M accepts: $L(M_w) \neq \emptyset \Leftrightarrow$ M accepts w.

2. Problems About TMs and Rice's Theorem (sl. 72)

Every non-trivial question about the language that a TM accepts is undecidable (Rice): whether L(M) is empty, finite, regular, context-free, … 'Non-trivial' = some TMs have the property and some do not. Proof sketch: let P be a non-trivial property of RE languages and (w.l.o.g.) $\emptyset$ does not have P; pick M0 with L(M0) having P. From <M, w> build M′: on input x first simulate M on w; if M accepts, run M0 on x and accept iff M0 does. Then L(M′) = L(M0) if M accepts w (P holds) and L(M′) = $\emptyset$ if not. A decider for P would therefore decide A_TM.

> **EXAM-READY ANSWER** Is 'the language accepted by a TM is palindromic' decidable? (2008-09 Q4(b)) No: the property is non-trivial (a TM accepting $\emptyset$ or {0} has it; one accepting {01} does not) and it is a property of the LANGUAGE, not of the machine, so Rice's theorem applies — a decider would decide A_TM.

3. Other Undecidable Problems (sl. 73–77)

- **Post's Correspondence Problem (PCP)**: two lists A = w1…wk and B = x1…xk of strings; a solution is a sequence i1 … im with w_i1 … w_im = x_i1 … x_im. Example (sl. 75): m = 4, i = (2, 1, 1, 3): w2w1w1w3 = x2x1x1x3 = 101111110. PCP is undecidable (sl. 76).
- Undecidable for CFGs (sl. 77): ambiguity of a CFG; whether L(G1) = L(G2); L(G1) = L(R) for a regular expression R; $L(G1) \subseteq L(G2)$; $L(R) \subseteq L(G1)$.
- Decidable for CFGs: membership (CYK), emptiness (mark generating variables).

Practice — Lesson 22

(a) Actual — 2015-16 Q7(b). (b) Actual — 2008-09 Q4(a)–(b). (c) Constructed.

(a) Suppose you know a problem P is undecidable. You want to prove another problem Q undecidable. Utilise P and the concept of reduction to prove Q is undecidable too. (10⅓)

(b) Describe and sketch a proof of Rice's theorem; is 'L(M) is palindromic' decidable? (12 + 3⅓)

(c) Show that E_TM = {<M> | L(M) = $\emptyset$} is undecidable by a reduction from A_TM.

✓ Solution

(a) EXAM-READY ANSWER in §1.

(b) Statement and sketch §2; the palindrome property: EXAM-READY ANSWER in §2.

(c) Given <M, w> build M_w that on input x ignores x, simulates M on w and accepts iff M accepts. L(M_w) = Σ^* if M accepts w and $\emptyset$ otherwise. A decider for E_TM applied to <M_w> answers 'M accepts w?' (accept iff E_TM says non-empty), deciding A_TM — impossible.

Quick Notes — Undecidability

Revision points for Lessons 19–22.

L19 · Classes

- Recursive = decidable (halts always); RE = recognised (may loop on non-members); non-RE = no TM.
- Decidable: A_DFA/NFA/REX, E_DFA, EQ_DFA, A_CFG, E_CFG. Undecidable: A_TM, HALT, E_TM, EQ_TM, EQ_CFG, ambiguity, PCP.
- Mars language {s} decidable; connected graph decidable (mark and propagate).

L20 · Counting and L_d

- Rationals countable (diagonals p+q); infinite binary sequences uncountable (flip the diagonal).
- TMs countable (coded as binary strings) but languages uncountable ⇒ non-RE languages exist.
- i-th string: 1w = i in binary (13th = 101). Rule code 0^i10^j10^k10^l10^m; machine c1 11 c2 11 …
- L_d = {w_i | w_i ∉ L(M_i)} not RE: M_i accepts w_i ⇔ w_i ∉ L_d — contradiction.

L21 · L_u, HALT, A_TM

- L recursive ⇒ complement recursive; L and complement RE ⇒ L recursive (run both in parallel).
- L_u RE (universal TM: code tape, simulated tape 0^i, state tape) but not recursive (else L_d RE).
- HALT RE (accept when M halts) not recursive (reduction from L_u). A_TM undecidable: D does the opposite of H on <M,<M>>.

L22 · Reductions

- Reduce the KNOWN undecidable problem TO the new one. Rice: non-trivial language properties of TMs are undecidable.
- PCP undecidable; CFG equality/ambiguity/inclusion undecidable.

Key Diagrams to Reproduce

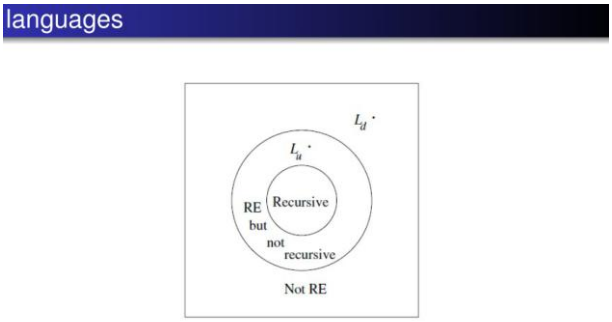


Figure 9.2: Relationship between the recursive, RE, and non-RE languages

Classes of languages (sl. 10)